Hdtg: An R Package for High-Dimensional Truncated Normal Simulation

by Zhenyu Zhang, Andrew Chin, Akihiko Nishimura and Marc A. Suchard

Abstract Simulating from the multivariate truncated normal distribution (MTN) is required in various statistical applications yet remains challenging in high dimensions. Currently available algorithms and their implementations often fail when the number of parameters exceeds a few hundred. To provide a general computational tool to efficiently sample from high-dimensional MTNs, we introduce the **hdtg** package that implements two state-of-the-art simulation algorithms: harmonic Hamiltonian Monte Carlo (Harmonic-HMC) and zigzag Hamiltonian Monte Carlo (Zigzag-HMC). Both algorithms exploit analytical solutions of the Hamiltonian dynamics under a quadratic potential energy with hard boundary constraints, leading to rejection-free methods. We compare their efficiencies against another state-of-the-art algorithm for MTN simulation, the minimax tilting accept-reject sampler (MET). The run-time of these three approaches heavily depends on the underlying multivariate normal correlation structure. Zigzag-HMC and Harmonic-HMC both achieve 100 effective samples within 3,600 seconds across all tests with dimension ranging from 100 to 1,600, while MET has difficulty in several high-dimensional examples. We provide guidance on how to choose an appropriate method for a given situation and illustrate the usage of **hdtg**.

1 Introduction

Sampling from a multivariate truncated normal (MTN) distribution is a recurring problem in many statistical applications. The MTN distribution of a d-dimensional random vector $\boldsymbol{x} \in \mathbb{R}^d$ has the form

$$\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma) \text{ with } (\mathbf{F}\boldsymbol{x} + \mathbf{g})_i \geq 0 \text{ for } i = 1, \ldots, m, \tag{1}$$

where $\boldsymbol{\mu}$ and Σ are the mean vector and covariance matrix, the $m \times d$ matrix $\mathbf{F}$ and m-dimensional vector $\mathbf{g}$ specify the m linear constraints and $(\cdot)_i$ denotes the ith vector element (Pakman and Paninski, 2014). MTNs arise in various contexts including probit and tobit models (Albert and Chib, 1993; Tobin, 1958), latent Gaussian models (Bolin and Lindgren, 2015), copula regression (Pitt et al., 2006), spatial models (Tsionas and Michaelides, 2016; Baltagi et al., 2018; Zareifard and Khaledi, 2021), Bayesian metabolic flux analysis (Heinonen et al., 2019), and many others. When the dimension d is small, a standard rejection sampler (Geweke, 1991; Kotecha and Djuric, 1999) works well and is a common choice. However, simulation from a larger MTN with hundreds or thousands of correlated dimensions remains a computational challenge. Work towards this goal includes harmonic Hamiltonian Monte Carlo (Pakman and Paninski, 2014, Harmonic-HMC), rejection sampling based on minimax (saddle point) exponential tilting (Botev, 2017, MET), and the most recent Zigzag Hamiltonian Monte Carlo (Nishimura et al., 2020, 2025, Zigzag-HMC) methods.

The MET method provides independent samples but can suffer from low acceptance rates and becomes impractical with $d > 100$, except in special cases like when the MTN has a strongly positive correlation structure (Botev, 2017). Both Harmonic-HMC and Zigzag-HMC are Markov chain Monte Carlo (MCMC) approaches that generate correlated samples, but can nonetheless be highly efficient and scale to thousands or more dimensions. To our knowledge, however, there is no general-purpose implementation of either method; the **tmg** package provided by Pakman and Paninski (2014) is no longer available on CRAN, and Zhang et al. (2023) implement Zigzag-HMC for their phylogenetics applications in the specialized software BEAST (Suchard et al., 2018). Therefore, we have developed the **hdtg** R package for efficient MTN simulation. The package implements tuning-free Zigzag-HMC and Harmonic-HMC. We provide performance comparisons among these two methods and a MET implementation from the **TruncatedNormal** package (Botev and Belzile, 2021). In most of the test cases with $d > 100$, Harmonic-HMC and Zigzag-HMC outperform MET. We then conclude with some empirical guidance on which method to use in different scenarios.

2 Algorithm

We begin by briefly introducing Harmonic-HMC and Zigzag-HMC, both of which are variants of HMC, an effective proposal generation mechanism exploiting the properties of Hamiltonian dynamics (Neal, 2011). Harmonic-HMC and Zigzag-HMC follow the same general framework. To sample $\boldsymbol{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$ from the target distribution $\pi_X(\boldsymbol{x})$, the HMC variants introduce an auxiliary *momentum* variable $\boldsymbol{p}$ and define an augmented target distribution $\pi(\boldsymbol{x}, \boldsymbol{p}) = \pi_X(\boldsymbol{x})\, \pi_P(\boldsymbol{p})$ in the joint

space. They then propose the next state by first re-sampling the momentum variable from its marginal and then simulating the solution of Hamiltonian dynamics governed by the differential equations

$$\frac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t} = \nabla K(\boldsymbol{p}), \quad \frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}t} = -\nabla U(\boldsymbol{x}), \tag{2}$$

where $U(\boldsymbol{x}) = -\log \pi_X(\boldsymbol{x})$ and $K(\boldsymbol{p}) = -\log \pi_P(\boldsymbol{p})$ are referred to as *potential* and *kinetic* energies. The dynamics are simulated for a pre-set time duration T and the end state constitutes a valid Metropolis proposal to be accepted or rejected according to the standard formula (Metropolis et al., 1953; Hastings, 1970).

The most common versions of HMC use the momentum distribution $\pi_P(\boldsymbol{p}) \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and rely on the leapfrog integrator to numerically solve (2), as its solutions are analytically intractable in general settings. Harmonic-HMC takes advantage of the fact that (2) admits analytical solutions when the target $\pi_X(\boldsymbol{x})$ is an MTN. The solution follows independent harmonic oscillations along the principal components of the covariance/precision matrix (Pakman and Paninski, 2014); we thus refer to the algorithm as Harmonic-HMC. Truncation boundaries are handled via elastic "bounces" against hard potential energy walls (Neal, 2011). Algorithm 1 provides pseudo-code for Harmonic-HMC following Pakman and Paninski (2014).

Zigzag-HMC differs from the common HMC versions in that it deploys a Laplace momentum (Nishimura et al., 2020, 2025)

$$\pi_P(\boldsymbol{p}) \propto \prod_{i=1}^{d} \exp(-|p_i|). \tag{3}$$

The Hamiltonian dynamics then become

$$\frac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t} = \mathrm{sign}(\boldsymbol{p}), \quad \frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}t} = -\nabla U(\boldsymbol{x}), \tag{4}$$

where $\mathrm{sign}(p_i)$ returns 1 if p_i is positive and -1 otherwise. Because the velocity $\mathrm{d}\boldsymbol{x}/\mathrm{d}t \in \{\pm 1\}^d$ remains constant until one of the p_i flips its sign, the trajectory of these Hamiltonian dynamics has a zigzag pattern, hence the name Zigzag-HMC. The zigzag dynamics also admit analytical solutions under an MTN target and can handle the truncation in the same manner as in Harmonic-HMC. Algorithm 2 provides pseudo-code for Zigzag-HMC. Our current Zigzag-HMC implementation is limited to the coordinate-wise bounds $\boldsymbol{l} \leq \boldsymbol{x} \leq \boldsymbol{u}$. We refer interested readers to Nishimura et al. (2025) and Zhang et al. (2023) for more details.

The simulation duration T, i.e. how long Hamiltonian dynamics is simulated for each proposal generation, critically affects efficiencies of both Harmonic and Zigzag-HMC. For Harmonic-HMC, Pakman and Paninski (2014) suggest setting $T = \pi/2$; when using this fixed T, however, we observe inefficiencies in some of our examples in Section 2.4 due to Hamiltonian dynamics' periodic behaviors (Neal, 2011). We therefore randomize the duration T, as recommended by Neal (2011), and draw it from a uniform distribution on $[\pi/8, \pi/2]$. For Zigzag-HMC, we adopt the choice $T = \sqrt{2}\lambda_{\min}^{-1/2}$ based on the heuristics of Nishimura et al. (2025), where $\lambda_{\min}$ is the minimal eigenvalue of the precision matrix $\Omega = \Sigma^{-1}$. We compute $\lambda_{\min}$ using the Lanczos algorithm (Demmel, 1997) as in the **mgcv** package (Wood, 2017). We further implement the no-U-turn sampler (NUTS) of Hoffman and Gelman (2014) to automatically determine the integration time. With NUTS, we only need to pick a base integration time ΔT which we set to $0.1\lambda_{\min}^{-1/2}$ as recommended by Nishimura et al. (2025).

Algorithm 1 Harmonic-HMC for MTNs

Require: $\boldsymbol{x}, T, \boldsymbol{\mu}, \Omega, \mathbf{F}, \mathbf{g}$,

(1) Whitening:

$R \leftarrow \text{Cholesky}(\Omega)$

$\boldsymbol{z} \leftarrow R(\boldsymbol{x} - \boldsymbol{\mu})$

(2) Momentum draw:

$\boldsymbol{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

(3) Dynamics:

$\tau \leftarrow T$

while $\tau > 0$ **do**

  % Find next boundary event

  $(t_b, j_b) \leftarrow \text{NEXTBOUNCE}(\boldsymbol{z}, \boldsymbol{p}, \mathbf{F}, \mathbf{g})$

  % The event time is bounded by τ

  $t \leftarrow \min\{t_b, \tau\}$

  % Advance time t:

  $\boldsymbol{z} \leftarrow \boldsymbol{p}\sin(t) + \boldsymbol{z}\cos(t)$

  $\boldsymbol{p} \leftarrow \boldsymbol{p}\cos(t) - \boldsymbol{z}\sin(t)$

  % Update $\boldsymbol{p}$ for a boundary event

  % $\mathbf{F}_{j_b}$ is the j_bth row vector of $\mathbf{F}$

  if $t < \tau$ **then**

    $\alpha \leftarrow \dfrac{\mathbf{F}_{j_b} \cdot \boldsymbol{p}}{\mathbf{F}_{j_b} \cdot \mathbf{F}_{j_b}}$

    $\boldsymbol{p} \leftarrow \boldsymbol{p} - 2\alpha \mathbf{F}_{j_b}^{\top}$

  end if

  $\tau \leftarrow \tau - t$

end while

(4) Unwhiten:

$\boldsymbol{x} \leftarrow R^{-1}\boldsymbol{z} + \boldsymbol{\mu}$

return $\boldsymbol{x}$

Function NEXTBOUNCE($\boldsymbol{z}, \boldsymbol{p}, \mathbf{F}, \mathbf{g}$):

$t_{\text{b}} \leftarrow \infty$, $j_{\text{b}} \leftarrow -1$

for $j = 1$ **to** m **do**

  % $(\mathbf{F}\boldsymbol{p})_j$ is the jth element of $\mathbf{F}\boldsymbol{p}$

  $f_a \leftarrow (\mathbf{F}\boldsymbol{p})_j$, $f_b \leftarrow (\mathbf{F}\boldsymbol{z})_j$

  $u_j \leftarrow \sqrt{f_a^2 + f_b^2}$

  if $u_j > |g_j|$ **then**

    $\varphi_j \leftarrow -\text{atan2}(f_a, f_b)$

    $A \leftarrow \arccos(-g_j/u_j)$

    $t_1 \leftarrow -\varphi_j + A$

    $t_2 \leftarrow -\varphi_j - A + 2\pi$

    if $t_1 < 0$ **then**

      $t_{\text{c}} \leftarrow t_2$

    else

      $t_{\text{c}} \leftarrow \min(t_1, t_2)$

    end if

    if $t_{\text{c}} < t_{\text{b}}$ **then**

      $t_{\text{b}} \leftarrow t_{\text{c}}$, $j_{\text{b}} \leftarrow j$

    end if

  end if

end for

return $(t_{\text{b}}, j_{\text{b}})$

Algorithm 2 Zigzag-HMC for MTNs

Require: $\boldsymbol{x}$, T, $\boldsymbol{\mu}$, Ω

(1) Momentum draw:

$p_i \sim \text{Laplace}(u = 0, b = 1)$

$\boldsymbol{v} \leftarrow \text{sign}(\boldsymbol{p})$

(2) Pre-compute:

$\boldsymbol{\varphi}_{\boldsymbol{x}} \leftarrow \Omega(\boldsymbol{x} - \boldsymbol{\mu})$

$\boldsymbol{\varphi}_{\boldsymbol{v}} \leftarrow \Omega\boldsymbol{v}$

(3) Dynamics:

$\tau \leftarrow T$

while $\tau > 0$ **do**

  % Find gradient event time t_g

  $\boldsymbol{a} \leftarrow \boldsymbol{\varphi}_{\boldsymbol{v}}/2$, $\boldsymbol{b} \leftarrow \boldsymbol{\varphi}_{\boldsymbol{x}}$, $\boldsymbol{c} \leftarrow -\boldsymbol{p}$

  $t_g \leftarrow \min_i\{\text{minPositiveRoot}(a_i, b_i, c_i)\}^*$

  %Find boundary event time t_b

  for $i = 1$ **to** d **do**

    if $v_i > 0$ **then**

      $t_i \leftarrow U_i - x_i$

    else

      $t_i \leftarrow x_i - L_i$

    end if

  end for

  $t_b \leftarrow \min_i t_i$

  %The actual event happens at time t

  $t \leftarrow \min\{t_g, t_b, \tau\}$

  % Advance time t

  $\boldsymbol{x} \leftarrow \boldsymbol{x} + t\boldsymbol{v}$

  $\boldsymbol{p} \leftarrow \boldsymbol{p} - t\boldsymbol{\varphi}_{\boldsymbol{x}} - t^2\boldsymbol{\varphi}_{\boldsymbol{v}}/2$

  $\boldsymbol{\varphi}_{\boldsymbol{x}} \leftarrow \boldsymbol{\varphi}_{\boldsymbol{x}} + t\boldsymbol{\varphi}_{\boldsymbol{v}}$

  % Update momentum

  if a gradient event happens at i_g **then**

    $v_{i_g} \leftarrow -v_{i_g}$

  else if a boundary event happens at i_b **then**

    $v_{i_{\text{b}}} \leftarrow -v_{i_{\text{b}}}$, $p_{i_{\text{b}}} \leftarrow -p_{i_{\text{b}}}$

  end if

  $\boldsymbol{\varphi}_{\boldsymbol{v}} \leftarrow \boldsymbol{\varphi}_{\boldsymbol{v}} + 2v_i\Omega\boldsymbol{e}_i$

  % $\boldsymbol{e}_i$ has 1 in position i and 0 elsewhere

  $\tau \leftarrow \tau - t$

end while

return $\boldsymbol{x}$

*minPositiveRoot(a_i, b_i, c_i) returns the minimal positive root of the equation $a_i t^2 + b_i t + c_i = 0$, or returns $+\infty$ if no positive root exists.

The algorithmic divergence between the two methods stems from their momentum distributions. Harmonic-HMC employs Gaussian momentum, generating harmonic oscillatory dynamics via trigonometric functions. In contrast, Zigzag-HMC uses Laplace momentum, producing piecewise-linear trajectories with constant velocity updates. This distinction leads to differences in event detection: Harmonic-HMC computes boundary bounce times using trigonometric relationships, while Zigzag-HMC solves quadratic equations for gradient events and checks coordinate distances for boundary events (Algorithm 1 and 2). These algorithmic differences lead to varying opportunities for implementation optimization, as summarized in Table 1. The piecewise-linear dynamics of Zigzag-HMC, consisting primarily of vector additions and multiplications, map well to single instruction multiple data (SIMD) instructions, which can process multiple values simultaneously within a single CPU core. In contrast, Harmonic-HMC's trigonometric operations (sin, cos, atan2, arccos) are inherently less SIMD-friendly due to their transcendental nature. While both samplers are implemented in optimized C++ with Eigen for linear algebra, Zigzag-HMC's algorithmic structure is inherently more amenable to the low-level hardware optimizations (SIMD and aligned memory) that yield performance gains on modern processors. When examining the computational performance benchmarks in Section 2.4, it is important to recognize that the observed speed differences stem from both algorithmic efficiencies and the hardware-specific optimizations detailed here.

Table 1: Comparison of implementation optimizations between Harmonic-HMC and Zigzag-HMC

Optimization	Harmonic-HMC	Zigzag-HMC
Language	C++ via Rcpp	C++ via Rcpp
Linear algebra	Eigen library	Eigen library
RNG[1]	`std::mt19937`	`std::mt19937`
SIMD[2]	No	Yes (SSE/AVX)
Memory optimization	Standard Eigen	Aligned allocation

[1]RNG: random number generator. [2]SIMD: single instruction multiple data.

In addition to Harmonic-HMC and Zigzag-HMC, the **hdtg** package also implements the Markovian-zigzag sampler (Bierkens et al., 2019) for MTNs. The Markovian-zigzag belongs to another emerging class of MCMC algorithms that are based on piecewise deterministic Markov processes (PDMP) (Fearnhead et al., 2018). As established by Nishimura et al. (2025), Markovian-zigzag is a close cousin of the Hamiltonian-based Zigzag-HMC, with the key difference being the amount of retained momentum information during sampling. Markovian-zigzag exhibited lower sampling efficiency compared to Harmonic-HMC and Zigzag-HMC, particularly on targets with correlated parameters (Nishimura et al., 2025). Therefore, we do not include Markovian-zigzag in our performance comparisons of Section 2.4. The Markovian-zigzag function is nevertheless available in the package for users who wish to experiment with it.

3 Using hdtg

The **hdtg** package allows users to draw MCMC samples from an MTN. As an example, one may use the following code to generate 1,000 samples from a 10-dimensional MTN with zero mean and an identity covariance matrix truncated to the positive orthant:

```
# set the random seed
set.seed(1)
# draw MTN samples using Zigzag-HMC
samplesZHMC <- zigzagHMC(nSample = 1000, mean = rep(0, 10), prec = diag(10),
                         init = rep(0.1, 10), lowerBounds = rep(0, 10),
                         upperBounds = rep(Inf, 10))
# draw MTN samples using Harmonic-HMC
samplesHHMC <- harmonicHMC(nSample = 1000, mean = rep(0, 10), choleskyFactor = diag(10),
                           precFlg = TRUE, init = rep(0.1, 10),
                           constrainDirec = diag(10), constrainBound = rep(0, 10))
```

The arguments are:

- `nSample`: number of samples.
- `mean`: a d-dimensional mean vector.
- `prec`: the precision matrix.

- `init`: a vector of the initial value that must satisfy all constraints.
- `lowerBounds`: a d-dimensional vector specifying the lower bounds.
- `upperBounds`: a d-dimensional vector specifying the upper bounds.
- `choleskyFactor`: upper triangular matrix $\mathbf{U}$ from Cholesky decomposition of precision or covariance matrix into $\mathbf{U}^T\mathbf{U}$.
- `precFlg`: whether `choleskyFactor` is from precision (`TRUE`) or covariance matrix (`FALSE`).
- `constrainDirec`: the $\mathbf{F}$ matrix.
- `constrainBound`: the $\mathbf{g}$ vector.

In addition to applications with fixed $\boldsymbol{\mu}$ and Ω, hierarchical modeling may require sampling these parameters from their respective conditional distributions, such as the phylogenetics example (Zhang et al., 2023) where their posterior distributions are of scientific interest. In such "random $\boldsymbol{\mu}$ or Ω" settings, one may call `zigzagHMC` or `harmonicHMC` inside an MCMC loop, with $\boldsymbol{\mu}$ and Ω updated at each iteration. For Zigzag-HMC, a more efficient approach is to reuse the existing C++ engine object which already stores the truncation boundaries and SIMD configuration and simply pass the updated $\boldsymbol{\mu}$ and Ω to the sampler. The example below illustrates the 10-dimensional MTN case with a random mean and precision. Note that the stationary distribution of this toy example is the joint distribution of both the MTN parameters (`m`, `prec`) and the truncated MTN variable itself. While the conditional distributions are standard (normal for `m`, Wishart for `prec`, MTN for the variable), their joint distribution is not a standard named distribution and does not admit a closed-form expression.

```
set.seed(1)
n <- 1000
d <- 10
samples <- array(0, c(n, d))
# initialize MTN mean and precision
m <- rnorm(d, 0, 1)
prec <- rWishart(n = 1, df = d, Sigma = diag(d))[,,1]

# call createEngine once
engine <- createEngine(dimension = d, lowerBounds = rep(0, d),
upperBounds = rep(Inf, d), seed = 1, mean = m, precision = prec)

HZZtime <- sqrt(2) / sqrt(min(mgcv::slanczos(A = prec, k = 1,
                         kl = 1)[['values']]))

currentSample <- rep(0.1, d)
for (i in 1:n) {
   m <- rnorm(d, 0, 1)
   prec <- rWishart(n = 1, df = d, Sigma = diag(d))[,,1]
   setMean(engine = engine, mean = m)
   setPrecision(engine = engine, precision = prec)
   currentSample <- getZigzagSample(position = currentSample, nutsFlg = F,
                                    engine = engine, stepSize = HZZtime)
   samples[i, ] <- currentSample
}
```

4 Efficiency comparison and method choice

To assess the performance of Harmonic-HMC, Zigzag-HMC and MET, we compare them on MTNs with a variety of correlation structures. The three examples are: 1) MTNs with its covariance matrix Σ drawn from the uniform LKJ distribution (Lewandowski et al., 2009) as implemented in the `rlkjcorr` function from package **trialr** (Brock, 2020); 2) MTNs with a compound symmetric covariance matrix such that $\Sigma_{i,i} = 1$ and $\Sigma_{i,j} = 0.9$ for $i \neq j$; and 3) a real-world MTN that arises as a posterior conditional distribution in a statistical phylogenetics model of HIV evolution (Zhang et al., 2021, 2023). For simplicity, we assume the truncation $x_i > 0$ for $i = 1, \ldots, d$ in the first two examples. For the HIV example, the truncation is determined by the signs of observed binary biological features.

We now specify our comparison criteria and the rationale behind them. A more efficient MCMC algorithm takes shorter time to achieve a certain effective sample size (ESS). For all three samplers considered, we compare their run-time to obtain the first 1, 100, or 1000 effectively independent samples (t_1, t_{100}, t_{1000}). We include all three metrics because t_{100} and t_{1000} reflect a practical run-time

for simulation from a fixed MTN and t_1 better captures the pre-processing overhead that remains relevant in cases where Σ is random. Recall that the main pre-processing costs are the Cholesky decomposition of Σ or Ω (Harmonic-HMC), calculating the minimal precision matrix eigenvalue $\lambda_{\min}$ (Zigzag-HMC), and solving the minimax optimization problem (MET). Therefore we have

$$\begin{aligned} t_1 &= t_0 + c, \\ t_{100} &= t_0 + 100c, \\ \text{and } t_{1000} &= t_0 + 1000c, \end{aligned} \tag{5}$$

where t_0 and c are the pre-processing time required for each Σ update and the average run-time per one effective sample. For simulation from a fixed MTN, t_0 is a one-time cost and so t_{100} or t_{1000} serve as better efficiency criteria. When Σ is random (e.g. the second example in Section 2.3), if Σ changes its value k times, the total run-time to obtain one effective sample for each Σ is kt_1 and so the t_1 criterion would be more informative.

For Harmonic-HMC and Zigzag-HMC, we estimate the ESS using the **coda** package (Plummer et al., 2006) and define n_1 as the average number of MCMC iterations required for one effectively independent sample. We approximate n_1 by $L/\text{ESS}_{\min}$, where $\text{ESS}_{\min}$ is the minimal ESS across all dimensions and L is the chain length. We fix $n_1 = 1$ for MET as it generates independent samples. Therefore c in Equation (5) equals the average time to complete n_1 iterations after pre-processing. Table 2 reports our efficiency comparison in terms of t_1, t_{100}, and t_{1000}. We run each test on an Apple M1 Pro equipped machine with 16GB of memory. Code and data to reproduce the results are available on Zenodo at https://doi.org/10.5281/zenodo.18618209.

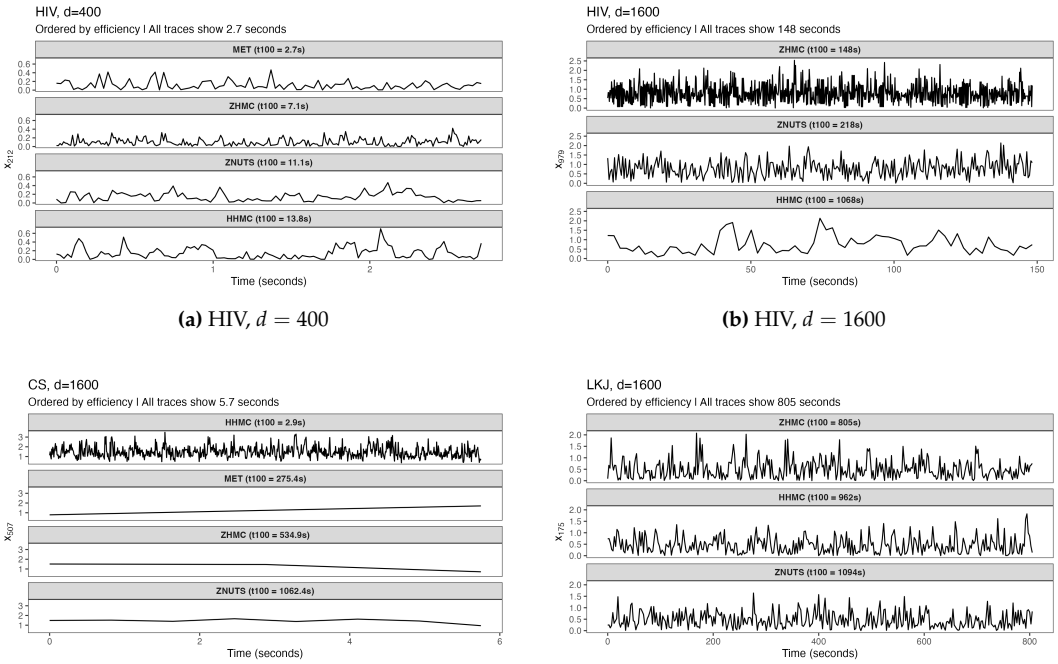
Table 2: Efficiency comparison of Harmonic-HMC, Zigzag-HMC, Zigzag-HMC with NUTS (Zigzag-NUTS), and MET sampling approaches across three example correlation structures. We report t_1, t_{100}, and t_{1000} (in seconds) — the run-times to obtain 1, 100, and 1000 effective samples, respectively. LKJ and HIV cases where MET takes more than two hours to generate 100 samples are omitted. We benchmark each test for three replications using **rbenchmark** (Kusnierczyk, 2012) and report the average run-time. Bold numbers are column minimums in each test.

		$d = 100$			400			1600		
		t_1	t_{100}	t_{1000}	t_1	t_{100}	t_{1000}	t_1	t_{100}	t_{1000}
LKJ	Harmonic-HMC	**0.004**	**0.21**	**2.1**	**0.11**	11	107	**8.0**	962	9680
	Zigzag-HMC	0.017	0.66	6.6	0.27	**9.3**	**92**	9.9	**805**	**8015**
	Zigzag-NUTS	0.014	0.60	6.1	0.29	10	102	13	1094	10891
	MET	1.9	19	101	–	–	–	–	–	–
$CS_{0.9}$	Harmonic-HMC	$<$ **0.001**	**0.014**	**0.15**	**0.007**	**0.086**	**0.83**	**0.42**	**2.9**	**25**
	Zigzag-HMC	0.006	0.39	3.9	0.21	18	180	6.3	535	5317
	Zigzag-NUTS	0.013	1.1	12	1.0	102	1014	13	1062	10610
	MET	0.087	0.12	0.33	4.4	4.9	7.2	272	275	313
HIV	Harmonic-HMC	**0.003**	0.32	3.4	0.17	14	138	10.7	1068	10968
	Zigzag-HMC	0.009	0.35	3.4	**0.14**	7.1	71	**2.8**	**148**	**1480**
	Zigzag-NUTS	0.013	0.66	6.5	0.17	11	109	4.3	218	2261
	MET	0.040	**0.056**	**0.19**	2.3	**2.7**	**5.1**	–	–	–

Dashes (–) indicate the method required 2 hours for 100 samples.

The efficiency of all three methods strongly depends on the correlation structure. MET fails to generate 100 effectively independent samples within two hours in a few higher dimensional tests, while Harmonic-HMC and Zigzag-HMC/NUTS enjoy a $t_{100} < 3600$ seconds across all tests. In the LKJ example, Zigzag-HMC/NUTS achieves comparable performance to Harmonic-HMC when d reaches 400. For high-dimensional LKJ and HIV ($d = 1600$) tests, Zigzag-HMC outperforms competing methods. Zigzag-NUTS is no more efficient than Zigzag-HMC across the tested examples. On the other hand, when Σ is compound symmetric with a high correlation of 0.9, Harmonic-HMC consistently outperforms the other methods. For MET, since solving the initial minimax optimization dominates its run-time, the cost scales sub-linearly with sample count ($t_{100} < 100t_1$ and $t_{1000} < 1000t_1$). This makes MET the preferred method when many effective samples are desired (as in HIV $d = 100, 400$ examples).

Figure 1 shows traceplots for four target MTNs from Table 2. Different methods excel on different targets, highlighting the importance of target-specific algorithm selection. In practice, we recommend running a quick efficiency comparison to decide which method to use. Nevertheless we provide some general guidance on method choice for high-dimensional MTN simulation:



(a) HIV, $d = 400$ **(b)** HIV, $d = 1600$

(c) CS, $d = 1600$ **(d)** LKJ, $d = 1600$

Figure 1: Time-normalized traceplots for four target MTNs using **ggplot2** (Wickham, 2016). The y-axis shows a randomly selected dimension.

- If $d \leq 100$ or the correlation structure is strongly positive, use MET or Harmonic-HMC. Harmonic-HMC may run faster but MET has the advantage of generating independent samples.
- For $d > 1000$, Zigzag-HMC/NUTS is presumably more efficient, though Harmonic-HMC may still outperform under strongly positive correlations.
- It is always worth trying MET which is free of MCMC convergence concerns. Since our simulation only examines a few correlation structures, it is possible that MET can handle other large MTNs.

A final point that needs consideration is that Zigzag-HMC/NUTS requires Ω and if only Σ is available, the method first inverts Σ. This is a one-time operation and likely negligible cost when Σ is constant. The approach does become expensive if Σ is random, as the $\mathcal{O}\left(d^3\right)$ inversion is necessary for each value of Σ. In practice, statistical models may be parameterized in terms of Σ (Lachaab et al., 2006; Molstad et al., 2021) or Ω (Baltagi et al., 2018; Lehnert et al., 2019; Li et al., 2020). Harmonic-HMC carries a similar limitation since it requires a $\mathcal{O}\left(d^3\right)$ Cholesky decomposition of Σ or Ω, whichever is provided. Therefore, when d is large and the target MTN has a random correlation structure, one may favor Zigzag-HMC/NUTS over Harmonic-HMC especially if a closed-form Ω is at hand.

5 Conclusion

This article introduces the **hdtg** package oriented for efficient MTN simulation. In most of our high-dimensional tests the implemented Harmonic-HMC and Zigzag-HMC algorithms outperform the current best approach available in the **TruncatedNormal** package. To our best knowledge, **hdtg** is the first tool that can generate samples from an arbitrary MTN with thousands of dimensions. We discuss the usage of functions and provide practical suggestions on method choice. We expect to see future large-scale statistical applications utilizing the efficiency of **hdtg**.

6 Acknowledgment

The **hdtg** package itself leverages several R infrastructure packages including **Rcpp** (Eddelbuettel and Balamuta, 2018), **RcppEigen** (Bates and Eddelbuettel, 2013), **RcppParallel**(Allaire et al., 2016), and **mgcv** (Wood, 2017) for efficient computation. It also incorporates external header-only libraries: `sse2neon.h` (DLTcollab and contributors, 2020) for ARM support and `span.h` (Brindle, 2018) for memory-safe array views. Research reported in this publication was supported by the National

Institute of General Medical Sciences and the National Institute of Allergy and Infectious Diseases of the National Institutes of Health under award numbers R35GM160458 (A.N.) and R01AI153044 (M.S.). The content is solely the responsibility of the authors and does not necessarily represent the official views of the National Institutes of Health.

References

J. H. Albert and S. Chib. Bayesian analysis of binary and polychotomous response data. *Journal of the American statistical Association*, 88(422):669–679, 1993. [p1]

J. Allaire, R. Francois, K. Ushey, G. Vandenbrouck, M. Geelnard, and H. Badr. Rcppparallel: Parallel programming tools for "rcpp". *R package version*, 4:20, 2016. [p7]

B. H. Baltagi, P. H. Egger, and M. Kesina. Generalized spatial autocorrelation in a panel-probit model with an application to exporting in China. *Empirical Economics*, 55(1):193–211, 2018. [p1, 7]

D. Bates and D. Eddelbuettel. Fast and elegant numerical linear algebra using the RcppEigen package. *Journal of Statistical Software*, 52(5):1–24, 2013. doi: 10.18637/jss.v052.i05. [p7]

J. Bierkens, G. O. Roberts, and P.-A. Zitt. Ergodicity of the zigzag process. *The Annals of Applied Probability*, 29(4):2266–2301, 2019. [p4]

D. Bolin and F. Lindgren. Excursion and contour uncertainty regions for latent Gaussian models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 77(1):85–106, 2015. [p1]

Z. Botev and L. Belzile. *TruncatedNormal: Truncated Multivariate Normal and Student Distributions*, 2021. URL https://CRAN.R-project.org/package=TruncatedNormal. R package version 2.2.2. [p1]

Z. I. Botev. The normal law under linear restrictions: simulation and estimation via minimax tilting. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 79(1):125–148, 2017. [p1]

T. Brindle. An implementation of c++20's std::span, 2018. URL https://github.com/tcbrindle/span. Boost Software License 1.0. Implementation of C++20's std::span (P0122R7) providing bounds-safe array views for C++14/17 compilers. [p7]

K. Brock. *trialr: Clinical Trial Designs in 'rstan'*, 2020. URL https://CRAN.R-project.org/package=trialr. R package version 0.1.5. [p5]

J. W. Demmel. *Applied numerical linear algebra*. SIAM, 1997. [p2]

DLTcollab and contributors. sse2neon: Sse to neon intrinsics translator, 2020. URL https://github.com/DLTcollab/sse2neon. Used version: 05-Jan-2021. MIT License. [p7]

D. Eddelbuettel and J. J. Balamuta. Extending R with C++: A Brief Introduction to Rcpp. *The American Statistician*, 72(1):28–36, 2018. doi: 10.1080/00031305.2017.1375990. [p7]

P. Fearnhead, J. Bierkens, M. Pollock, and G. O. Roberts. Piecewise deterministic markov processes for continuous-time Monte Carlo. *Statistical Science*, 33(3):386–412, 2018. [p4]

J. Geweke. Efficient simulation from the multivariate normal and student-t distributions subject to linear constraints and the evaluation of constraint probabilities. In *Computing science and statistics: Proceedings of the 23rd symposium on the interface*, pages 571–578. Citeseer, 1991. [p1]

W. K. Hastings. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, 1970. [p2]

M. Heinonen, M. Osmala, H. Mannerström, J. Wallenius, S. Kaski, J. Rousu, and H. Lähdesmäki. Bayesian metabolic flux analysis reveals intracellular flux couplings. *Bioinformatics*, 35(14):i548–i557, 2019. [p1]

M. D. Hoffman and A. Gelman. The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(1):1593–1623, 2014. [p2]

J. H. Kotecha and P. M. Djuric. Gibbs sampling approach for generation of truncated multivariate Gaussian random variables. In *1999 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings. ICASSP99 (Cat. No. 99CH36258)*, volume 3, pages 1757–1760. IEEE, 1999. [p1]

W. Kusnierczyk. *rbenchmark: Benchmarking routine for R*, 2012. URL https://CRAN.R-project.org/package=rbenchmark. R package version 1.0.0. [p6]

M. Lachaab, A. Ansari, K. Jedidi, and A. Trabelsi. Modeling preference evolution in discrete choice models: A Bayesian state-space approach. *Quantitative Marketing and Economics*, 4(1):57–81, 2006. [p7]

J. Lehnert, C. Kolbitsch, G. Wübbeler, A. Chiribiri, T. Schaeffter, and C. Elster. Large-scale Bayesian spatial-temporal regression with application to cardiac MR-perfusion imaging. *SIAM Journal on Imaging Sciences*, 12(4):2035–2062, 2019. [p7]

D. Lewandowski, D. Kurowicka, and H. Joe. Generating random correlation matrices based on vines and extended onion method. *Journal of multivariate analysis*, 100(9):1989–2001, 2009. [p5]

Z. R. Li, T. H. McComick, and S. J. Clark. Using Bayesian latent Gaussian graphical models to infer symptom associations in verbal autopsies. *Bayesian analysis*, 15(3):781, 2020. [p7]

N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. Equation of state calculations by fast computing machines. *Journal of Chemical Physics*, 21(6):1087–1092, 1953. [p2]

A. J. Molstad, L. Hsu, and W. Sun. Gaussian process regression for survival time prediction with genome-wide gene expression. *Biostatistics*, 22(1):164–180, 2021. [p7]

R. M. Neal. MCMC using Hamiltonian dynamics. In S. Brooks, A. Gelman, G. L. Jones, and X.-L. Meng, editors, *Handbook of Markov Chain Monte Carlo*, volume 2. CRC Press New York, NY, 2011. [p1, 2]

A. Nishimura, D. B. Dunson, and J. Lu. Discontinuous Hamiltonian Monte Carlo for discrete parameters and discontinuous likelihoods. *Biometrika*, 107(2):365–380, 2020. [p1, 2]

A. Nishimura, Z. Zhang, and M. A. Suchard. Zigzag path connects two Monte Carlo samplers: Hamiltonian counterpart to a piecewise deterministic markov process. *Journal of the American Statistical Association*, 120(550):1077–1089, 2025. [p1, 2, 4]

A. Pakman and L. Paninski. Exact Hamiltonian Monte Carlo for truncated multivariate Gaussians. *Journal of Computational and Graphical Statistics*, 23(2):518–542, 2014. [p1, 2]

M. Pitt, D. Chan, and R. Kohn. Efficient Bayesian inference for Gaussian copula regression models. *Biometrika*, 93(3):537–554, 2006. [p1]

M. Plummer, N. Best, K. Cowles, and K. Vines. Coda: Convergence diagnosis and output analysis for mcmc. *R News*, 6(1):7–11, 2006. URL https://journal.r-project.org/archive/. [p6]

M. A. Suchard, P. Lemey, G. Baele, D. L. Ayres, A. J. Drummond, and A. Rambaut. Bayesian phylogenetic and phylodynamic data integration using BEAST 1.10. *Virus Evolution*, 4(1):vey016, 2018. doi: 10.1093/ve/vey016. URL http://dx.doi.org/10.1093/ve/vey016. [p1]

J. Tobin. Estimation of relationships for limited dependent variables. *Econometrica: journal of the Econometric Society*, pages 24–36, 1958. [p1]

E. G. Tsionas and P. G. Michaelides. A spatial stochastic frontier model with spillovers: Evidence for italian regions. *Scottish Journal of Political Economy*, 63(3):243–257, 2016. [p1]

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ISBN 978-3-319-24277-4. URL https://ggplot2.tidyverse.org. [p7]

S. Wood. *Generalized Additive Models: An Introduction with R*. Chapman and Hall/CRC, 2 edition, 2017. [p2, 7]

H. Zareifard and M. J. Khaledi. A heterogeneous Bayesian regression model for skewed spatial data. *Spatial Statistics*, 46:100545, 2021. [p1]

Z. Zhang, A. Nishimura, P. Bastide, X. Ji, R. P. Payne, P. Goulder, P. Lemey, and M. A. Suchard. Large-scale inference of correlation among mixed-type biological traits with phylogenetic multivariate probit models. *The Annals of Applied Statistics*, 15(1):230–251, 2021. [p5]

Z. Zhang, A. Nishimura, N. S. Trovão, J. L. Cherry, A. J. Holbrook, X. Ji, P. Lemey, and M. A. Suchard. Accelerating bayesian inference of dependency between mixed-type biological traits. *PLoS computational biology*, 19(8):e1011419, 2023. [p1, 2, 5]

Zhenyu Zhang
Department of Biostatistics, Fielding School of Public Health,
University of California, Los Angeles,
650 Charles E. Young Dr. South, Los Angeles, CA
USA
ORCiD: 0000-0002-2176-2253
zyz606@ucla.edu

Andrew Chin
Department of Biostatistics, Bloomberg School of Public Health,
Johns Hopkins University,
615 North Wolfe Street, Baltimore, MD 21205
USA
ORCiD: 0009-0005-3832-841X
achin23@jhu.edu

Akihiko Nishimura
Department of Biostatistics, Bloomberg School of Public Health,
Johns Hopkins University,
615 North Wolfe Street, Baltimore, MD 21205
USA
ORCiD: 0000-0002-6932-2513
aki.nishimura@jhu.edu

Marc A. Suchard
Department of Human Genetics, David Geffen School of Medicine,
Department of Biomathematics, David Geffen School of Medicine,
Department of Biostatistics, Fielding School of Public Health,
University of California, Los Angeles,
695 Charles E. Young Drive, South, Los Angeles, CA
USA
ORCiD: 0000-0001-9818-479X
msuchard@ucla.edu